Design Google Drive

SystemsExpert design walkthrough

Clarifying questions

- Only file storage, or also live collaborative editing?
- Max file size? Versioning?
- Sharing permissions required?

Functional requirements

- Upload, download, delete files and folders
- Move/rename; list a folder
- Share with read/write permissions
- Sync across devices

Non-functional requirements

- Durability (never lose a file), high availability
- Large files resumable
- Consistent metadata (no ghost files)

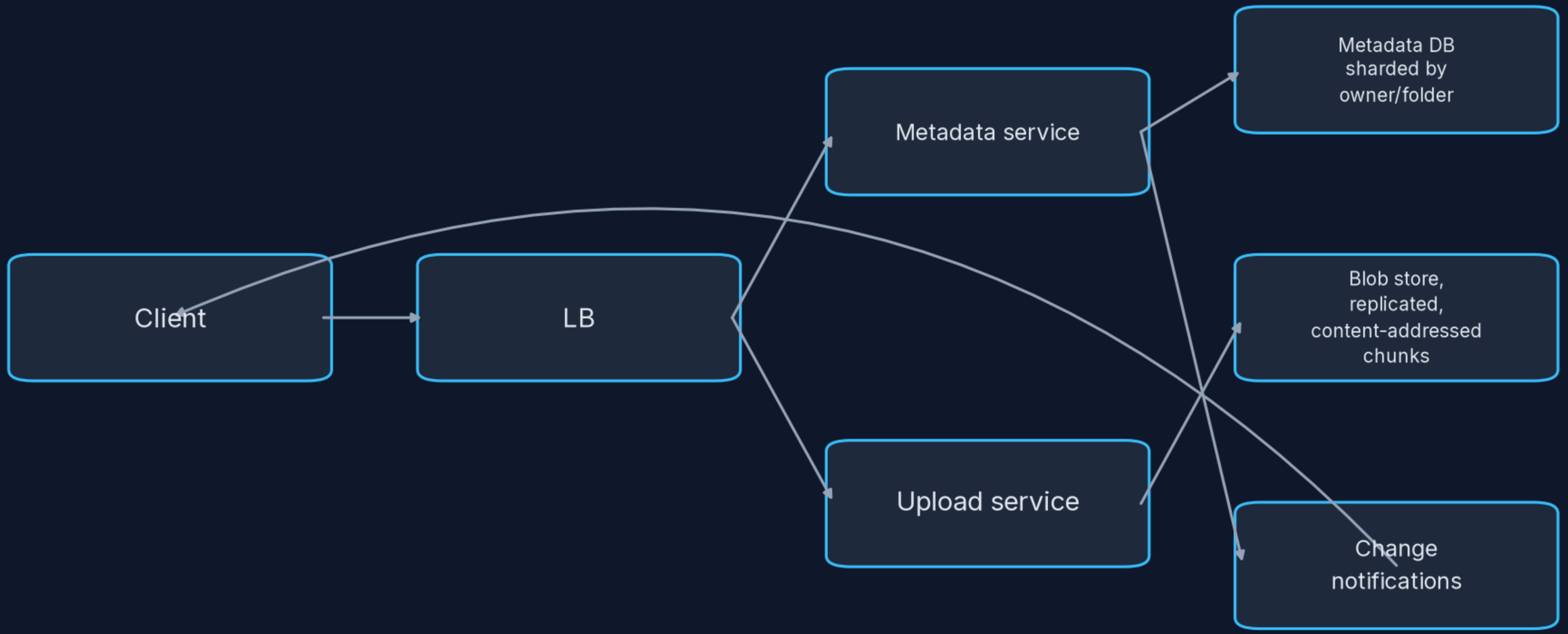
Capacity estimates

- ~1B users x ~15 GB avg used ~ exabytes of blobs
- Metadata: ~100 files/user → 100B rows; must be sharded
- Uploads chunked (e.g. 4-8 MB) for resumability and dedup

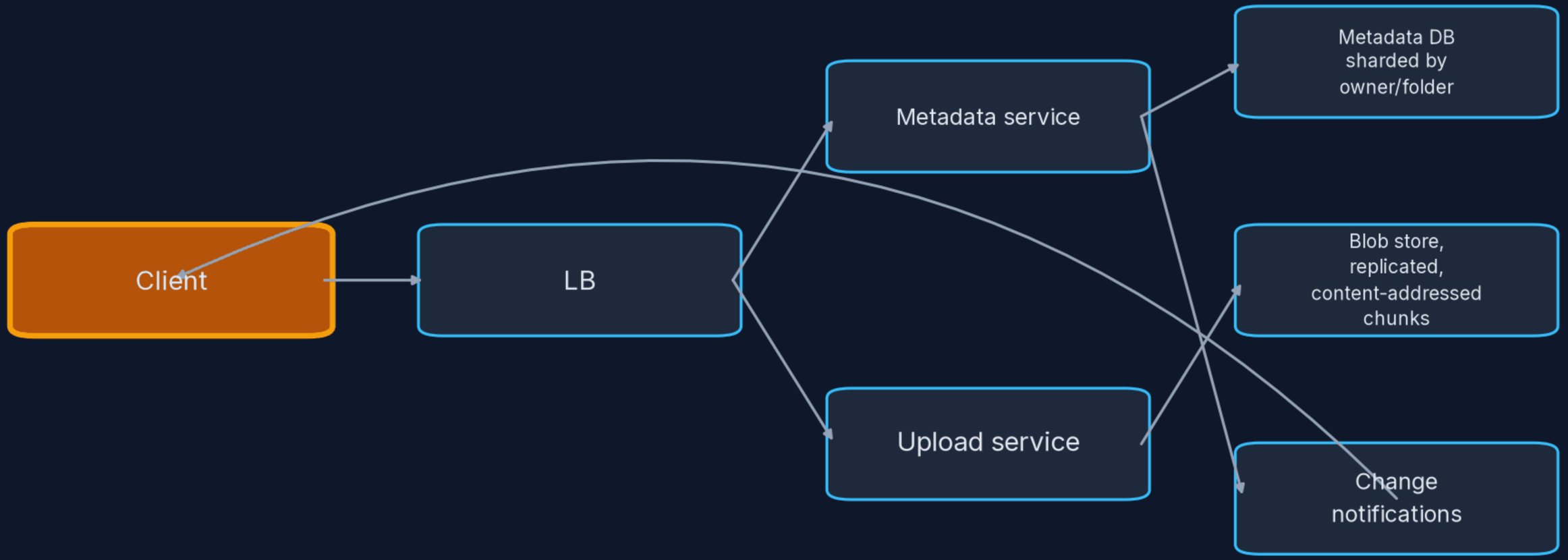
Capacity: worked out

- 1B users × 15 GB ≈ 15 EB of blobs
- × 3 replicas ≈ 45 EB raw (dedup helps)
- 100 files/user → 100B metadata rows
- ~1 KB/row → ~100 TB metadata → shard by owner

High-level architecture

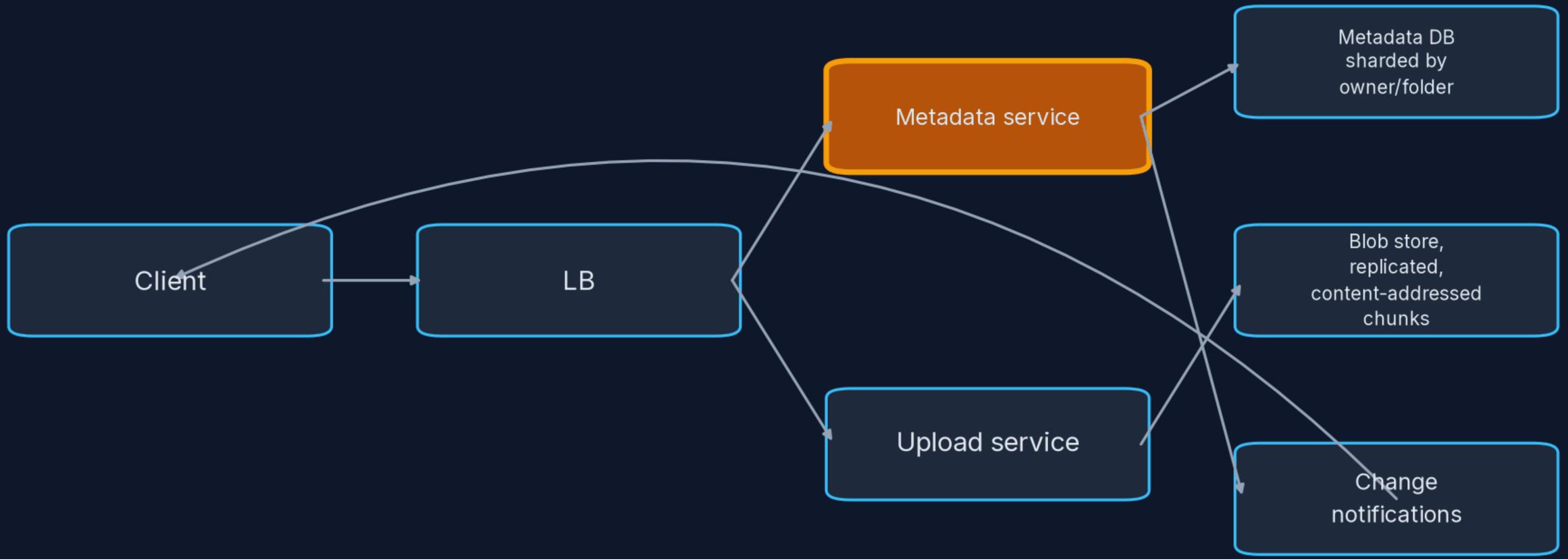


Request flow: Upload a file



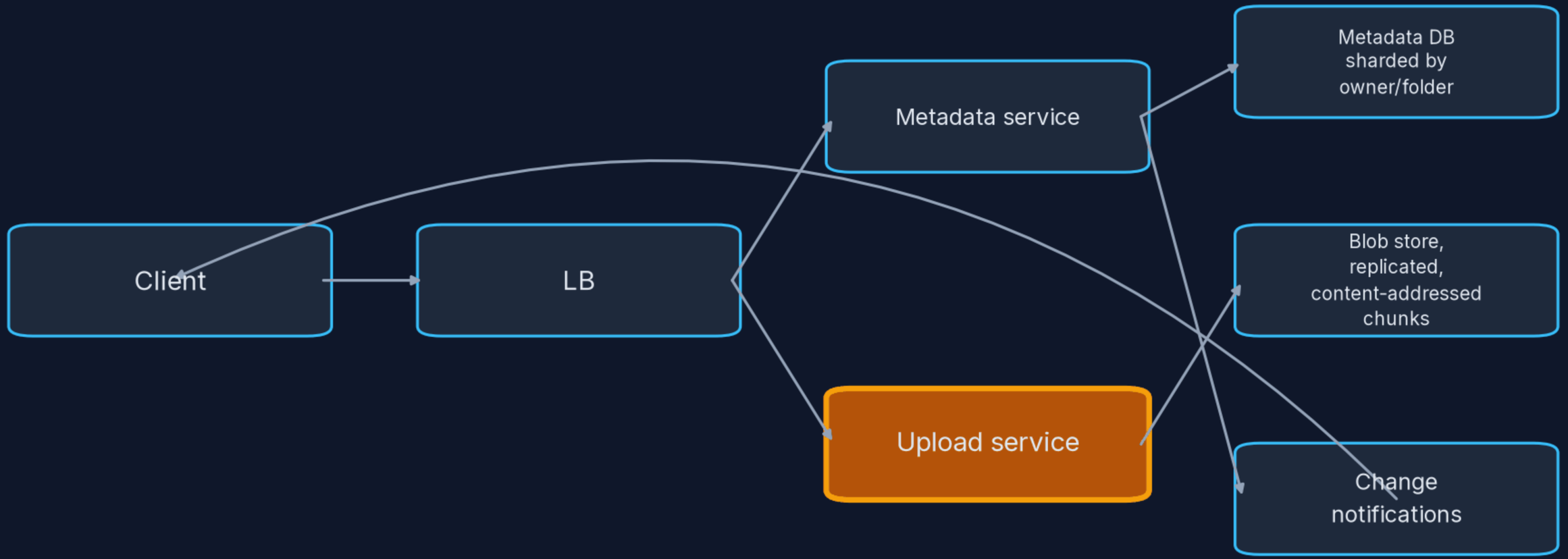
1. Client chunks the file

Request flow: Upload a file



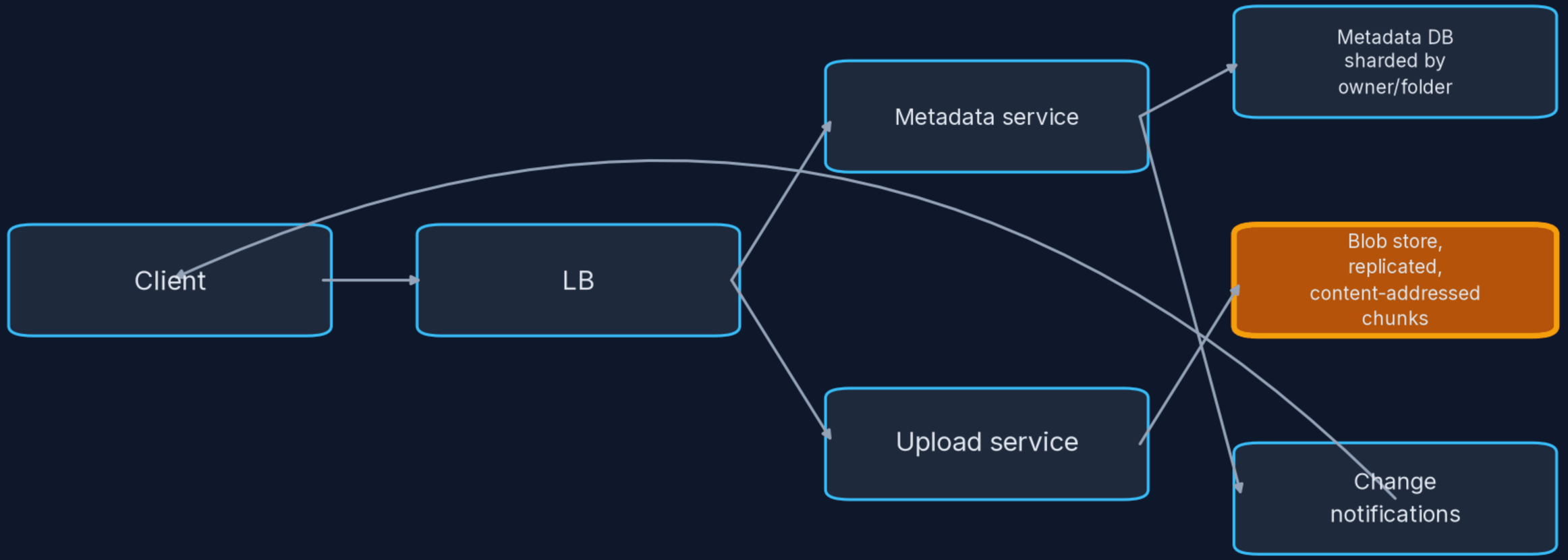
2. Ask which chunks are new

Request flow: Upload a file



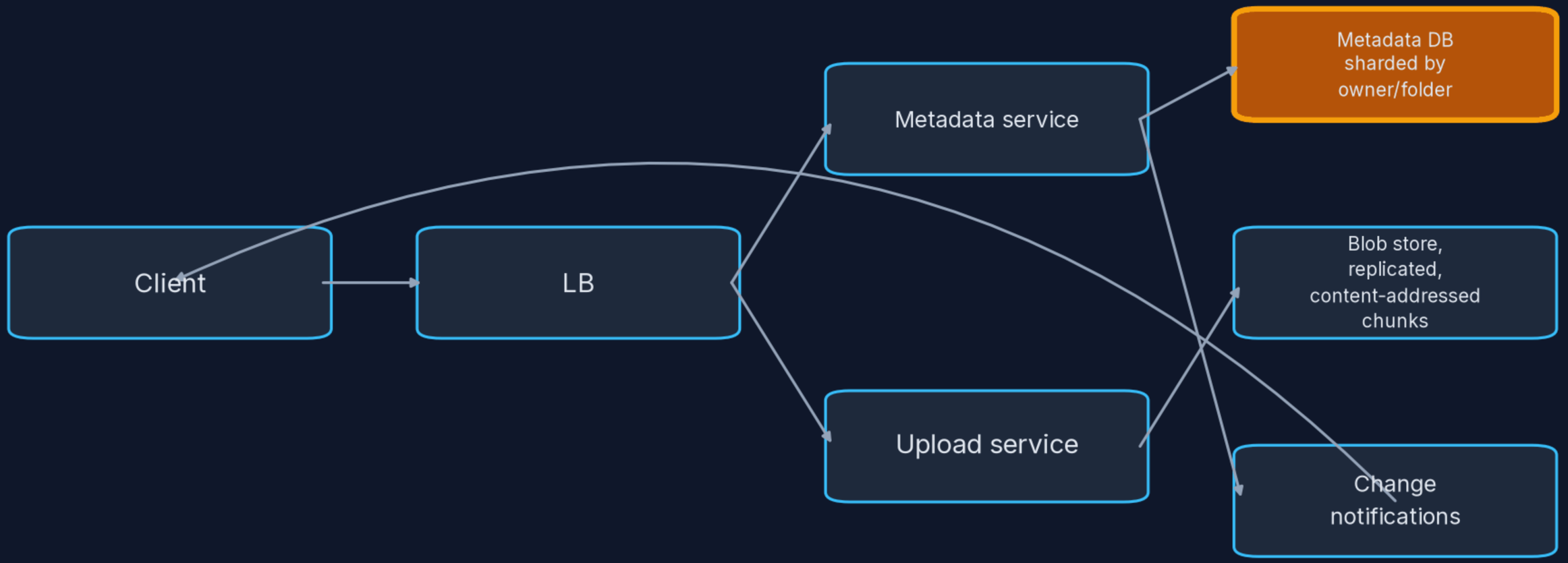
3. Upload only missing chunks

Request flow: Upload a file



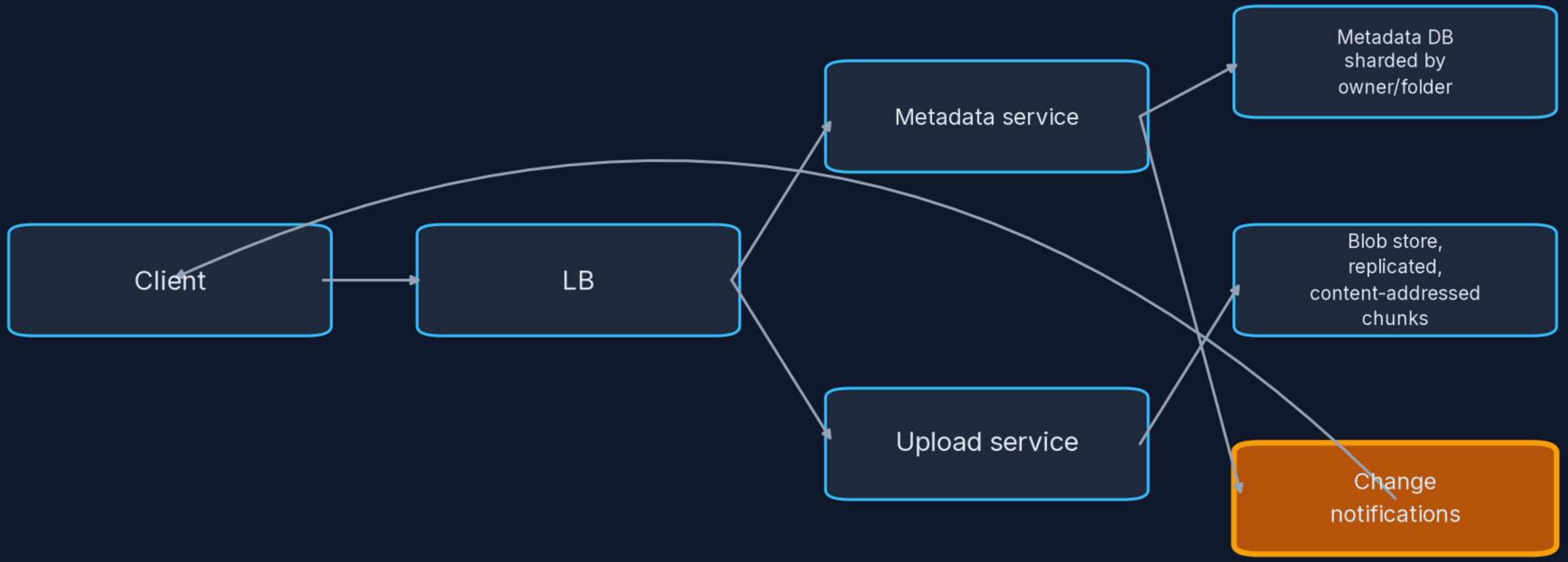
4. Chunks stored by hash

Request flow: Upload a file



5. Commit the new file version

Request flow: Upload a file



6. Other devices sync

Data model

- Files(id, owner_id, parent_id, name, is_folder, size, version, chunk_ids[], deleted)
- Acl(file_id, principal_id, role)
- Chunks(hash, size, ref_count, locations)

API

```
POST /files/upload-session {parentId, name, size} -> {sessionId}
PUT  /upload/{sessionId}/chunks/{n}
POST /upload/{sessionId}/commit -> file
GET  /files/{id}/content
GET  /folders/{id}/children
PATCH /files/{id} {name?, parentId?}
POST /files/{id}/share {principal, role}
```

Upload path

- Chunks are hashed client-side; the server skips chunks it already has (dedup).
- File metadata is committed only after all chunks are durable, so readers never see partial files.

Metadata consistency

- Folder moves must be atomic: keep a user's tree on one shard (shard by owner) so moves are single-shard transactions.

Sync

- Clients long-poll/websocket for change events with a per-user change log cursor; conflicts produce 'conflicted copy' files.

Key decisions

- Separate metadata from blobs → different scale and consistency needs
- Content-addressed chunks → dedup and resumable uploads
- Strongly consistent metadata → no lost edits
- Push change notifications → fast sync without polling

Trade-offs & alternatives

- Content-addressed chunks save space but complicate deletion (ref counts + GC)
- Sharding by owner makes shared-folder queries cross-shard
- Erasure coding cheaper than 3x replication for cold data

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Client, LB, Metadata service, Metadata DB sharded by owner/folder, Upload service
- Know your trade-offs and failure modes